PharmaSoft | Informe de seguridad S10

Lenguaje de Programación II · Unidad 2 · Sesión 4 · 18 de octubre de 2026

1. Objetivo y alcance

Se corrigió la conexión Oracle del proyecto AP9 y se implementó la actividad autónoma S10: autenticación JWT, permisos por rol, cambio de contraseña, asignación de roles y activación/desactivación de usuarios. También se integraron las pantallas Angular para usar estas funciones.

2. Entorno verificado

Componente	Configuración observada
Oracle	Oracle Free 23.26.1, contenedor oracle-local, host 1522 hacia contenedor 1521, servicio FREEPDB1, esquema SYSVENTAS.
Backend	Spring Boot 4, Java 24, API en http://localhost:8081/api/v1.
Frontend	Angular 22, http://localhost:4200; login, catálogo y pantalla de usuarios comprobados en Chrome.
Datos	69 categorías visibles desde la SPA; 5 cuentas de prueba de seguridad en desarrollo.

3. Método y resultados

Se ejecutaron 28 casos principales sobre HTTP real (16 base, 4 propios y 8 B1-B3), más 7 verificaciones complementarias. Se comparó el estado obtenido con el esperado y se guardó un JSON por ID sin tokens ni contraseñas. Las cuentas modificadas durante las pruebas fueron restauradas.

Grupo	Casos	Resultado
Autenticación A	4	4 correctos
Token T	5	5 correctos
Permisos P	4	4 correctos
Usuarios U	2	2 correctos
Conexión C	1	1 correcto
Propios X	4	4 correctos
Requerimientos B	8	8 correctos
Total	28	28/28 correctos

Evidencia principal: docs/matriz-seguridad-s10.md y docs/evidencias-s10/<ID>.json. El archivo Postman se entrega sin credenciales ni JWT reales.

Hallazgos y correcciones

4. Fichas de hallazgos

ID / severidad	Evidencia y causa	Corrección / estado
H-01 / Alta	El perfil dev apuntaba a localhost:1521, mientras Oracle del proyecto expone el contenedor por 1522. Ese puerto corresponde a otra instancia local.	URL dev cambiada a jdbc:oracle:thin:@localhost:1522/FREEPDB1. Hikari conectó a SYSVENTAS y C-01 devolvió categorías. Resuelto.
H-02 / Media	Un JWT emitido antes de cambiar la contraseña podía seguir vigente hasta expirar si solo se verificaba firma y fecha.	El token lleva una huella del hash BCrypt; el filtro la compara con la credencial actual. B-01a obtuvo 401 con el token anterior. Resuelto.
H-03 / Baja	El origen 127.0.0.1:4200 no estaba en la lista CORS inicial; el servidor Angular también puede iniciarse con ese host.	Se añadieron localhost y 127.0.0.1 para los puertos de desarrollo 4200/4201. Los orígenes de producción son configurables. Resuelto.

No quedan casos fallidos en la matriz ejecutada. El estado PASS demuestra los escenarios listados, no una auditoría exhaustiva de producción.

5. Diseño de los requerimientos nuevos

Requisito	Implementación	Evidencia
B1: PUT /auth/password	Usuario autenticado, validación 8-72 con letra y número, BCrypt, 204; 409 si la contraseña actual falla o se repite la nueva. El token previo queda revocado.	B-01, B-02, B-03, B-01a, B-03a
B2: PUT /usuarios/{id}/roles	ADMIN, conjunto de roles válido, 404 para ID inexistente, 409 si ADMIN se quita su propio rol; permisos recargados de BD por petición.	B-04, B-05, B-08, B-04a, B-05a
B3: PATCH /usuarios/{id}/estado	ADMIN, 404 si no existe, 409 si intenta desactivarse a sí mismo; token previo de usuario inactivo obtiene 401.	B-06, B-07, B-06a, B-07a

6. Revisión de frontend

La SPA usa inicio de sesión, interceptor Bearer, cierre de sesión ante 401, guardas de rutas y menú según permisos. Hay vistas para cambiar la contraseña y administrar roles/estado. El backend mantiene la autorización efectiva; ocultar enlaces en Angular solo mejora la navegación.

Análisis y reproducción

7. Preguntas de análisis de la guía

¿El login revela si falló el usuario o la contraseña? No. Credenciales erradas y cuenta deshabilitada responden 401 con mensaje uniforme: "Usuario o contraseña incorrectos" (A-03 y A-04).

¿Qué ocurre con un token anterior tras cambiar contraseña? El filtro verifica la huella de la contraseña almacenada en cada petición. Al cambiar el hash BCrypt, el token anterior devuelve 401 (B-01a); es necesario volver a iniciar sesión.

¿Qué pasa si se filtra la clave JWT? Quien la posea podría firmar tokens válidos. Debe rotarse JWT_SECRET desde un gestor de secretos o variable de entorno, reiniciar la API para invalidar firmas previas, revisar accesos y emitir nuevas sesiones. La clave y los tokens no se guardan en Git ni en Postman.

¿Basta proteger una URL o conviene proteger métodos? Las reglas por URL controlan el tráfico HTTP. La comprobación en métodos protege la operación si cambia el mapeo o se invoca desde otro controlador. El proyecto usa reglas por ruta y @PreAuthorize para reportes; las operaciones de usuarios están restringidas por URL. Conviene extender anotaciones de método a todo servicio crítico en una fase de endurecimiento.

8. Cómo reproducir

1. Iniciar oracle-local y comprobar el mapeo 1522:1521. Configurar DB_USERNAME, DB_PASSWORD y JWT_SECRET (Base64 de al menos 32 bytes).

2. En desarrollo, habilitar APP_SEED_DEMO_SECURITY=true y fijar PHARMA_DEMO_PASSWORD con una clave de prueba propia. Desde backend, ejecutar .\mvnw.cmd spring-boot:run.

3. En la raíz, ejecutar npm ci y npm start. Abrir http://localhost:4200, iniciar sesión y revisar Catálogo, Usuarios y Mi cuenta.

4. Ejecutar scripts/verify_s10.py con PHARMA_DEMO_PASSWORD definido. Revisar la matriz y sus JSON. La colección Postman requiere completar sus variables locales y ejecutar sobre datos de demostración.

9. Límites y próximos pasos

La prueba se hizo en el Oracle local del curso. El perfil dev usa Hibernate ddl-auto:update; para despliegue se recomienda una migración versionada. El cierre de sesión limpia el token del navegador, pero un JWT copiado sigue válido hasta caducar si no cambió la contraseña ni se desactivó la cuenta. No se probó una carga multiusuario ni un despliegue remoto.

La guía solicita trabajo en rama, colección, matriz e informe. Esta copia está en feature/seguridad-pruebas-tinoco. La entrega en B-Learning y el PR requieren acciones separadas; este informe registra solo el trabajo local verificado.

Las páginas siguientes forman un anexo visual: capturas de la aplicación en Chrome y vistas de fragmentos exactos de los archivos locales. Los JSON muestran respuestas guardadas de las pruebas HTTP; la imagen de Postman muestra la colección, no una ejecución adicional.

Anexo visual | Conexión y datos

Las vistas de código se generaron a partir de los archivos indicados en cada captura, con números de línea originales y fragmentos omitidos señalados. Las pantallas de la SPA se capturaron en Chrome sobre la instancia local.

Figura 1. Configuración dev: API 8081 y Oracle en localhost:1522/FREEPDB1.

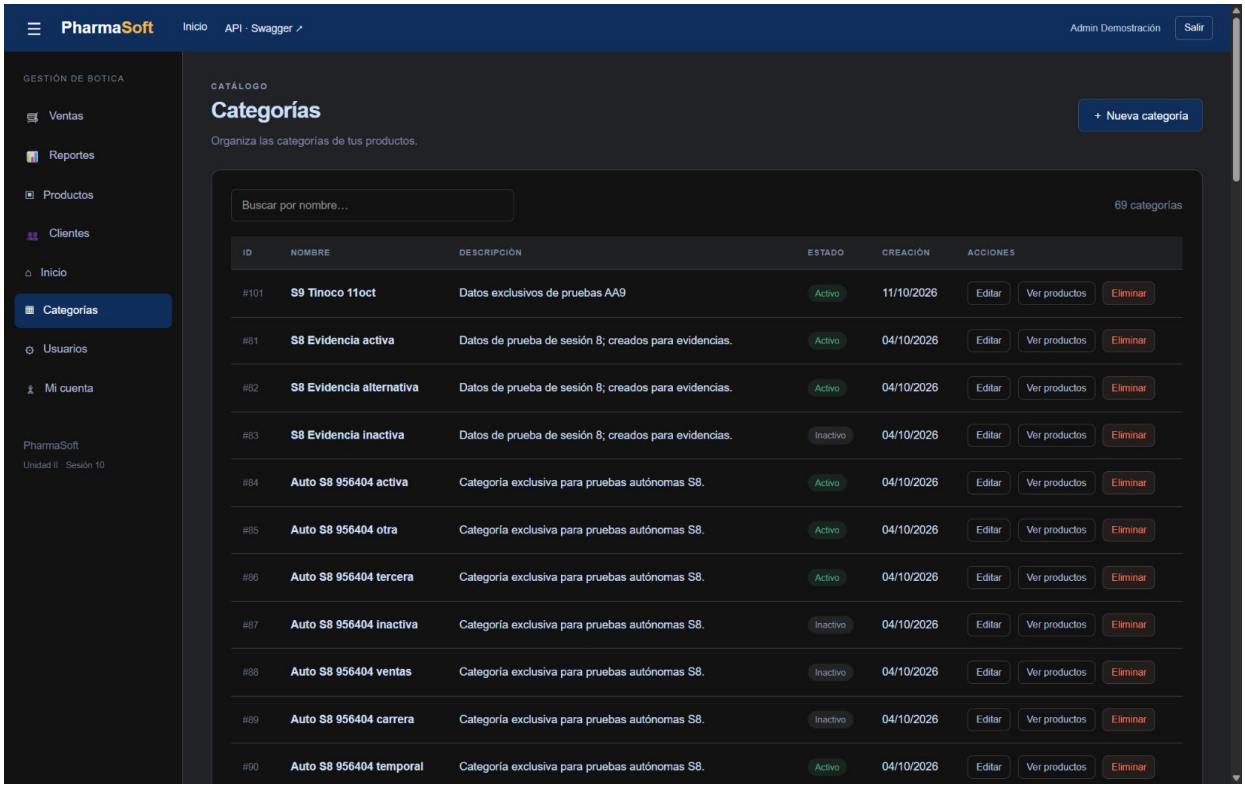
Conexión backend - Oracle y puertos

Vista de evidencia generada de archivos locales del proyecto · fragmentos con líneas originales

backend/src/main/resources/application-dev.yaml

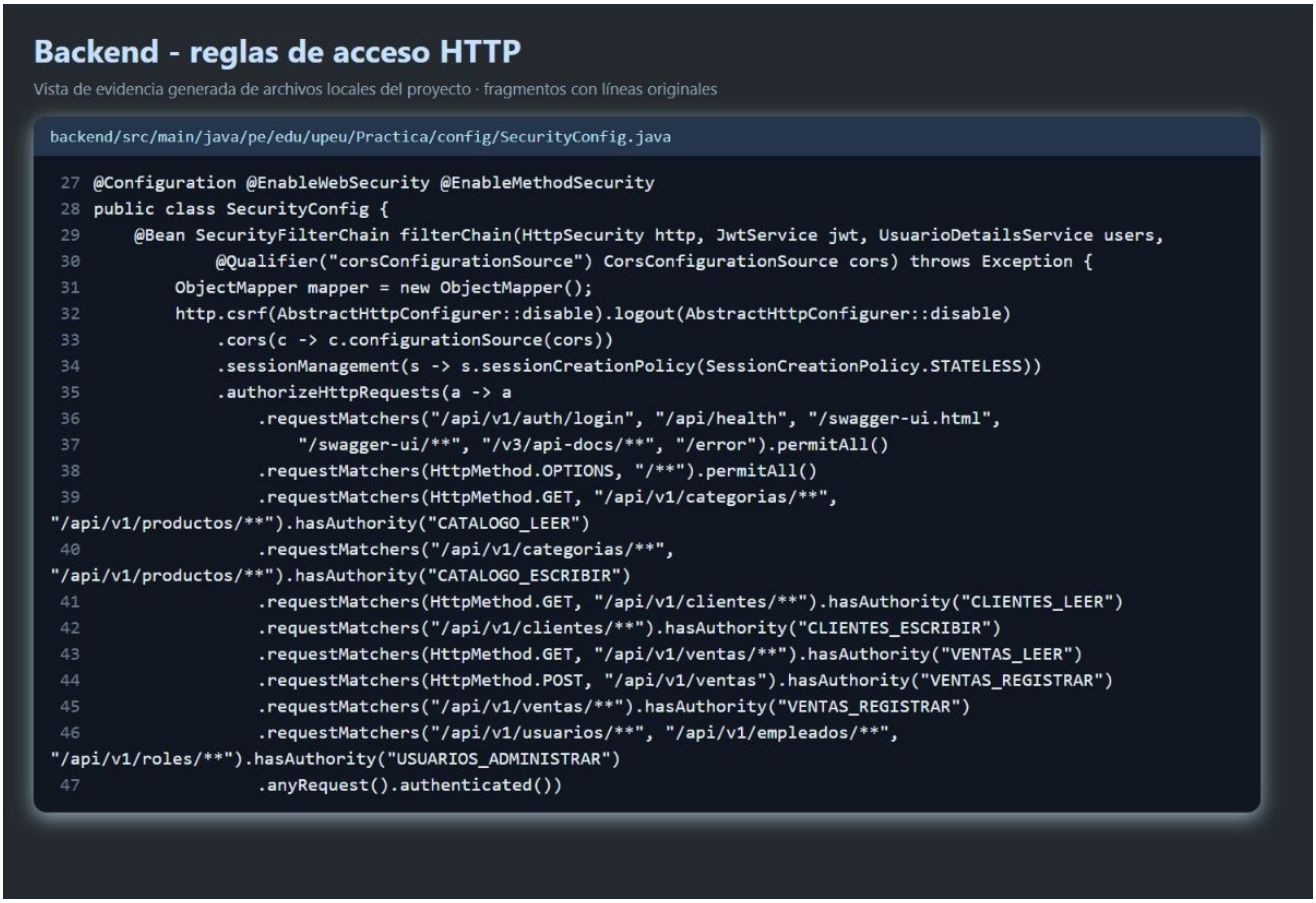
```
 1 server:
 2     port: 8081
 3 spring:
 4   datasource:
 5     url: ${DB_URL:jdbc:oracle:thin:@localhost:1522/FREEPDB1}
 6     username: ${DB_USERNAME}
 7     password: ${DB_PASSWORD}
 8   jpa:
 9     database-platform: org.hibernate.dialect.OracleDialect
10     hibernate:
11       ddl-auto: update
12       show-sql: true
... fragmento omitido
25 app:
26   jwt:
27     secret: ${JWT_SECRET}
28     expiracion-minutos: ${JWT_EXPIRACION_MINUTOS:60}
```

Figura 2. Categorías cargadas en la SPA: 69 registros desde Oracle.



Anexo visual | Seguridad del backend

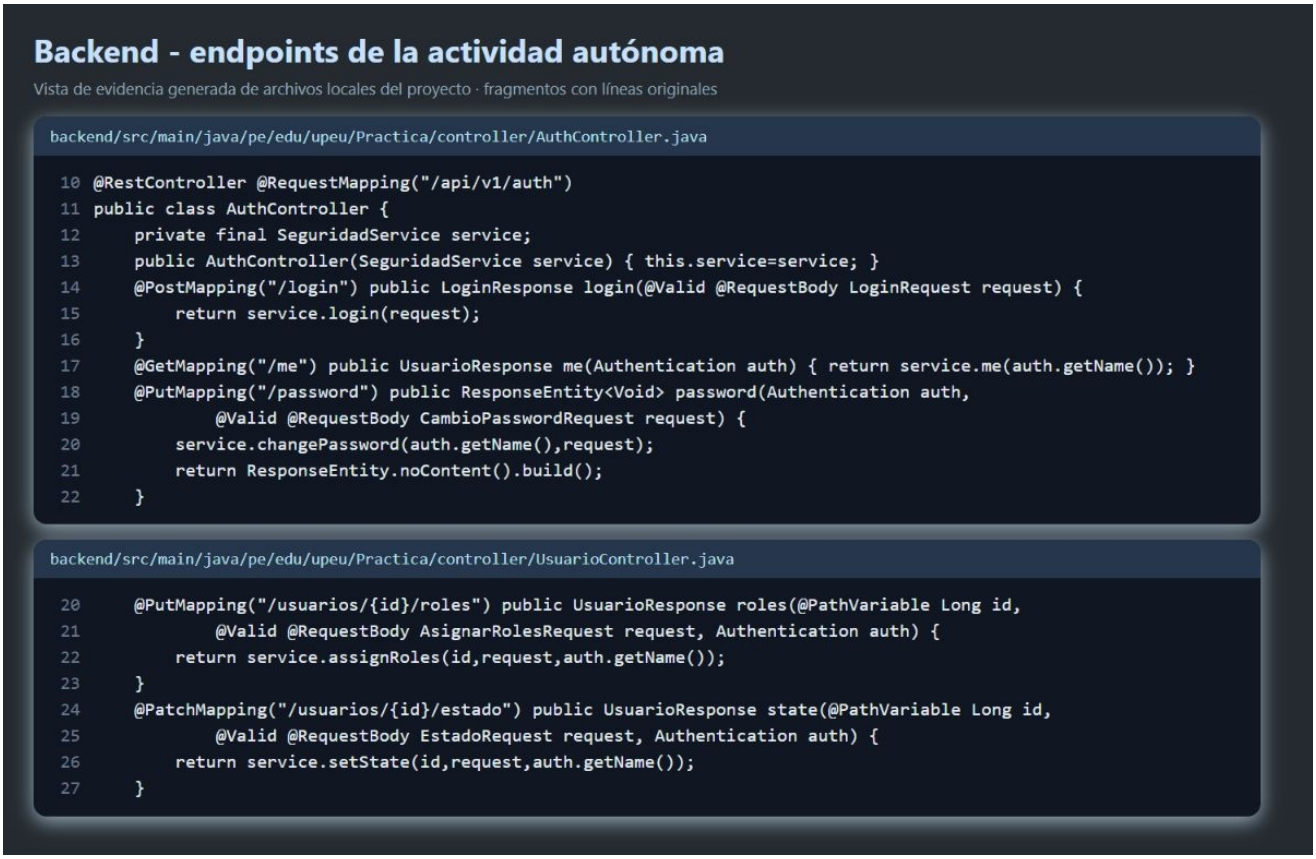
Figura 3. Reglas HTTP: JWT, permisos por operación, 401 y 403.



Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Endpoints de la actividad autónoma

Figura 4. Controladores para cambio de contraseña, roles y estado.



Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Reglas de negocio

Figura 5. BCrypt y protecciones para el administrador.

Backend - reglas de negocio B1, B2 y B3

Vista de evidencia generada de archivos locales del proyecto · fragmentos con líneas originales

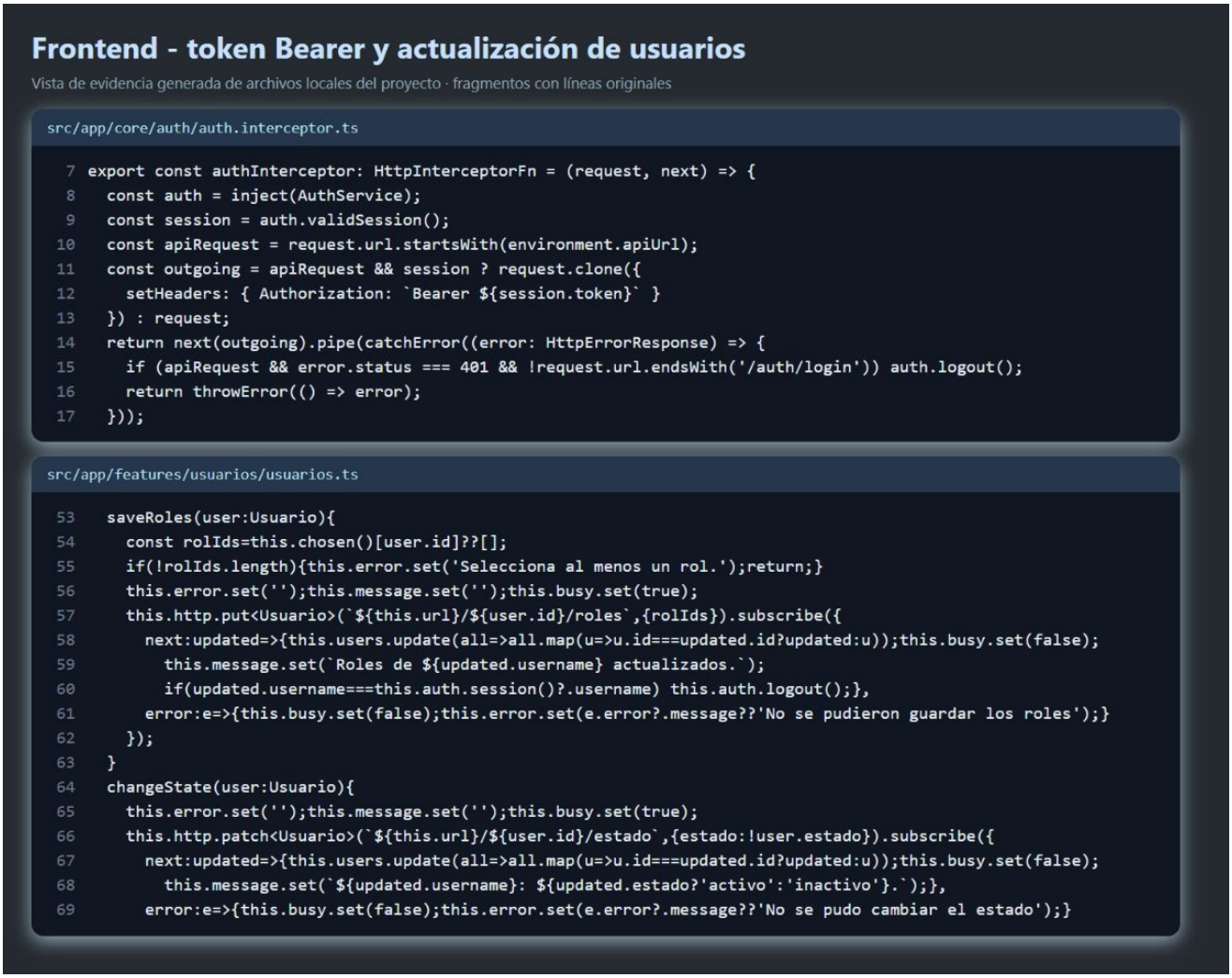
backend/src/main/java/pe/edu/upeu/Practica/service/impl/SeguridadService.java

```
 48     public void changePassword(String username, CambioPasswordRequest request) {
 49         Usuario user=byName(username);
 50         checkPasswordLength(request.passwordNueva());
 51         if (!encoder.matches(request.passwordActual(), user.getPassword()))
 52             throw new ReglaNegocioException("La contraseña actual no es correcta");
 53         if (encoder.matches(request.passwordNueva(), user.getPassword()))
 54             throw new ReglaNegocioException("La nueva contraseña debe ser distinta de la actual");
 55         user.setPassword(encoder.encode(request.passwordNueva()));
 56         log.info("Contraseña cambiada: usuario={}", user.getUsername());
... fragmento omitido
 86     public UsuarioResponse assignRoles(Long id, AsignarRolesRequest request, String actor) {
 87         Usuario user=byId(id); Set<Rol> selected=findRoles(request.rolIds());
 88         if (user.getUsername().equalsIgnoreCase(actor) && user.getRoles().stream().anyMatch(r ->
r.getNombre().equals("ADMIN"))
 89                 && selected.stream().noneMatch(r -> r.getNombre().equals("ADMIN")))
 90             throw new ReglaNegocioException("Un administrador no puede quitarse el rol ADMIN");
 91         user.setRoles(new HashSet<>(selected));
 92         return response(user);
 93     }
 94     @Transactional
 95     public UsuarioResponse setState(Long id, EstadoRequest request, String actor) {
 96         Usuario user=byId(id);
 97         if (!request.estado() && user.getUsername().equalsIgnoreCase(actor))
 98             throw new ReglaNegocioException("Un administrador no puede desactivarse a sí mismo");
 99         user.setEstado(request.estado());
100         return response(user);
```

Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Integración del frontend

Figura 6. Interceptor Bearer, cierre ante 401 y llamadas PUT/PATCH de usuarios.



Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Respuestas JSON verificadas

Figura 7. B-01a devuelve 401 con token previo y C-01 confirma 69 filas Oracle.



Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Colección Postman

Figura 8. Estructura JSON de la colección S10 y prueba de estado HTTP.



JSON - colección de Postman S10

Vista de evidencia generada de archivos locales del proyecto · fragmentos con líneas originales

postman/PharmaSoft-S10-Seguridad.json

```
{
  "info": {
    "name": "PharmaSoft - S10 Seguridad",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
    "description": "28 casos de la matriz S10. Use solo un entorno local de desarrollo. Los casos B alteran
usuarios demo; restaurar al final."
  },
  "item": [
    {
      "name": "A - Autenticación",
      "item": [
        {
          "name": "A-01 - Login ADMIN",
          "request": {
            "method": "POST",
            "header": [
              {
                "key": "Content-Type",
... fragmento omitido
          },
          "event": [
            {
              "listen": "test",
              "script": {
                "type": "text/javascript",
                "exec": [
                  "pm.test(\"A-01: HTTP 200\", () => pm.response.to.have.status(200));",
                  "if(pm.response.code===200) pm.collectionVariables.set(\"adminToken\",
pm.response.json().token);"
                ]
              }
            }
```

Fuente: archivo indicado dentro de la captura. La colección Postman usa variables locales; no se incluyen credenciales ni JWT reales.

Anexo visual | Pantallas de usuario

Figura 9. Administración de roles y estado de cinco cuentas de prueba.

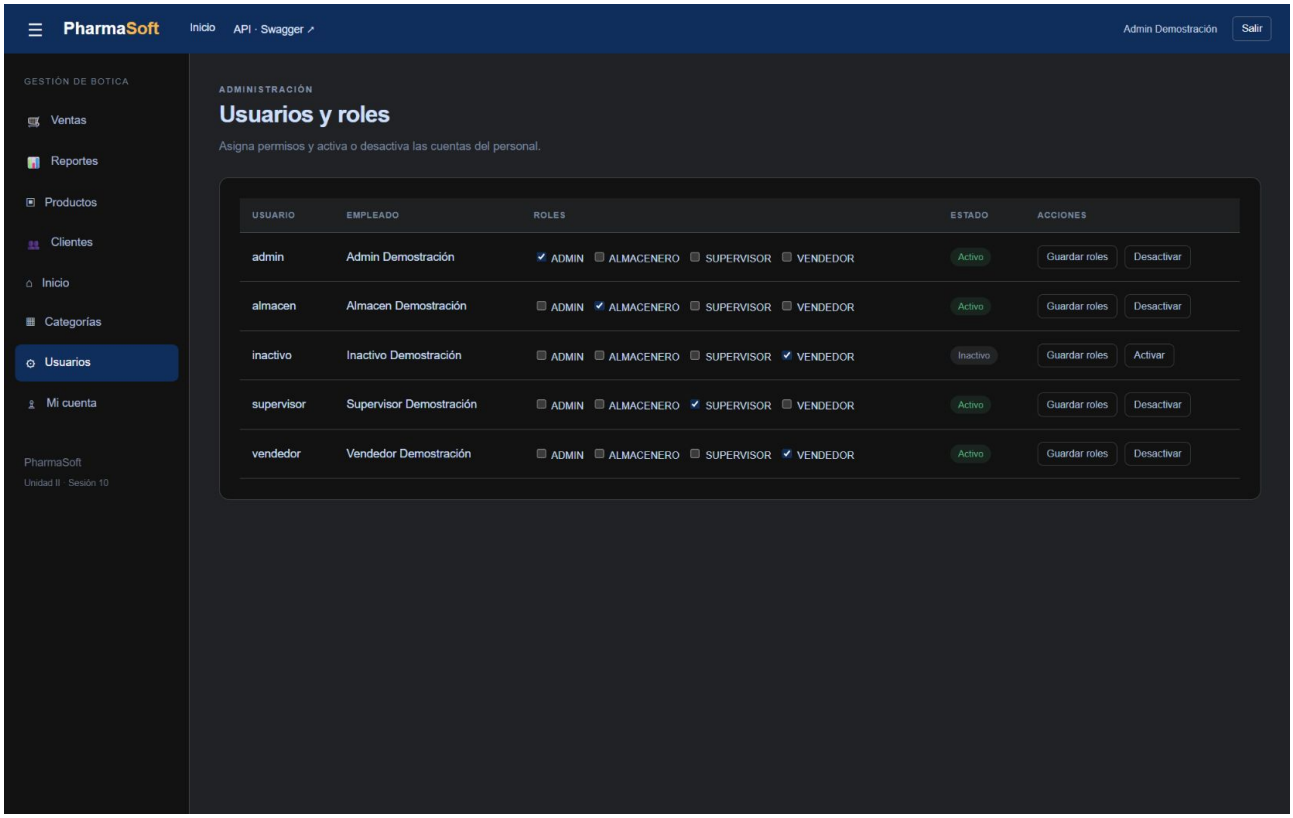


Figura 10. Vista Mi cuenta para el cambio de contraseña.

